// Q1) why we need the search = debounce((text)=>{},delay)?? why making complex

// concept of clouser
// Q2) Does the timer is static vaible declare once when debounce called (stored in heap)
// Q3) Timer is in debounce and delay is also in the debounce so here the bag theory is appleid right?
// Q4) How we will know when to use the clouser or function inside fucntion (here you said that's why i know)

Q1) Why do we need `search = debounce(callback, delay)`?

Because we want to **create the debounce function once and use it multiple times**.

JavaScript

```
const search = debounce(callback, 500);

search("V");
search("Vi");
search("Viraj");
```

- `debounce()` creates and returns a new function.
- `search` stores that returned function.
- Every `search()` call uses the **same timer variable**.

If we called `debounce()` separately for every keystroke, each call would create a different timer, so they couldn't cancel one another.

Q2) Is `timer` a static variable declared once when `debounce()` is called?

Almost yes, but technically it's not a static variable.

JavaScript

```
function debounce(callback, delay) {
  let timer;

  return function (text) {
    // Can access timer
  };
}
```

- `timer` is created once per `debounce()` call.
- It survives because the returned function remembers it.
- Its value can change every time `search()` executes.
- Its storage is managed by the JavaScript engine; closures don't guarantee a particular heap layout.

Calling `debounce()` again creates a **separate timer variable**.

Q3) Is the closure's bag theory applied here?

Yes! Exactly.

Remember our bag analogy?

When `debounce()` finishes, its returned function carries access to the variables it needs.

Closure bag for `search`

`timer`	Current timer handle
`delay`	500 ms
`callback`	Original function

Technically, the closure retains access to its lexical environment, not necessarily copies of these variables.

Q4) How do I recognize when to use a closure?

Ask yourself one question:

"Does this function need to remember something from an earlier function call?"

If yes, a closure may be useful.

Situation	Why closure?
Debounce	Remember previous timer
Counter	Remember previous count
API cache	Remember cached data privately
Event listener wrapper	Remember original callback
Rate limiter	Remember recent request history

Main rule: Use closures when a function needs to preserve access to variables after its outer function has finished.

You don't need closures for every function.

Remember: A closure is not something you manually activate. JavaScript creates it automatically when a function references variables from its surrounding scope.